

FORGOT PASSWORD, BACKEND (direct reset, no e-mail)

Spring Boot 3.2.5 / Java 17. **Replaces the earlier e-mail-link version.** No SMTP, no mail account, no new table, no new Maven dependency. Files: 1 service rewritten, 1 DTO file, 1 controller, 1 property line.

0. What the user experiences

1. Login page, click **Forgot password?**, type the e-mail, click **Continue**.
2. The same page asks for a **new password** (and the registered mobile number, only if you switch that on).
3. Click **Change password**. It is saved at once.
4. The user logs in with the new password. The old one stops working.

1. The ideas, from zero (tied to your code)

Why we cannot send the old password. Passwords in `users.password` are BCrypt hashes (`SecurityConfig.passwordEncoder`). A hash cannot be turned back into text, so the only possible fix is to **replace** it: `user.setPassword(encoder.encode(newPassword))`, then `userRepo.save(user)`. At login, `AuthController` compares what the person types with that hash, so the new password works immediately.

Two endpoints, both public (the person cannot log in yet, so `SecurityConfig` lists them in `permitAll`; `RateLimitFilter` counts them with the other login calls).

Endpoint	Body	Answer
POST <code>/api/forgot-password</code> (step 1)	<code>{email}</code>	<code>{requireMobile}</code> . It never says whether the account exists.
POST <code>/api/reset-password</code> (step 2)	<code>{email, newPassword, mobile?}</code>	200 "Your password has been changed", or 400 / 404 / 429 with a message

The honest security picture. With no e-mail, nothing proves that the person owns the account. **Anyone who knows an e-mail address can change that account's password.** That is acceptable for a college demo and is what you asked for, but it is not how a real product works (a real one e-mails a link or a code). So the service adds protection that costs nothing:

1. **ADMIN accounts are never reset here.** The answer is "Admin passwords cannot be reset here". Admin password recovery stays with the owner (`app.admin.reset-password` in `application.properties`).
2. **Rate limits** (`RateLimiter`, the same class OTP uses): 10 attempts per hour per e-mail, 20 per hour per device. A guesser gets HTTP 429 with `Retry-After`.
3. **Sessions end.** `RefreshTokenService.revokeAll(userId)` signs out every other device. An access token already issued lives at most 15 minutes (`jwt.access-expiration-ms`).

4. **Same password rules.** The server accepts 8 to 72 characters with a letter and a number (the Angular page enforces the stronger registration rules).
5. **Optional second detail:** `reset.require-mobile=true`. Step 1 then tells the page to show a "Registered mobile number" box, and step 2 changes the password only when that number matches `users.mobile_number` (spaces and dashes ignored). Accounts made with Google sign-up have the placeholder mobile `0000000000`, which is not a secret, so in this mode they are refused with "Use Continue with Google". It is **off by default**, exactly as you asked. Turn it on with `RESET_REQUIRE_MOBILE=true` before showing the project to anyone outside your team.

Unknown e-mail. Step 2 answers 404 "No account found for this e-mail. Please register first." (the same wording as Google login). Step 1 does not reveal it.

Old e-mailed links. The route `/reset-password` now redirects to `/forgot-password`.

2. What goes where

File	Job
<code>dto/ResetDtos.java</code>	request and answer records
<code>service/PasswordResetService.java</code>	all the rules above
<code>controller/PasswordResetController.java</code>	the two endpoints
<code>application.properties</code>	one line: <code>reset.require-mobile</code>

Delete these files if you used the e-mail version before: `model/PasswordResetToken.java`, `repository/PasswordResetTokenRepo.java`, `service/ResetMailer.java`, `service/SmtpResetMailer.java`. In `pom.xml` remove the `spring-boot-starter-mail` dependency. Remove the `spring.mail.*`, `mail.from`, `app.frontend-url`, `reset.token-minutes` and `reset.dev-expose-link` lines. The old table `password_reset_tokens` can stay or be dropped (`DROP TABLE password_reset_tokens;`).

3. Properties (add at the end of `application.properties`)

```
reset.require-mobile=${RESET_REQUIRE_MOBILE:false}
```

4. Check it

1. Register a customer, then open the login page, **Forgot password?**, type the e-mail, **Continue**, type a new strong password twice, **Change password**.
2. Log in with the new password: it works. The old one gives 401.
3. Try an e-mail that is not registered: 404 message and a "Create an account" link.
4. Try the admin e-mail: "Admin passwords cannot be reset here".
5. Set `RESET_REQUIRE_MOBILE=true`, restart: the page now asks for the mobile number; a wrong one is refused.

Tested: the Java logic with stand-in classes, **56 checks, 0 failures** (both modes, admin, Google placeholder, rate limits, weak passwords, unknown e-mail). The real Spring application and MySQL were not part of that test.

5. Complete code

springapp/src/main/java/com/examly/springapp/dto/ResetDtos.java

```
package com.examly.springapp.dto;

/** Request and response bodies of the forgot-password endpoints (direct reset: no e-mail is sent). */
public final class ResetDtos {
    private ResetDtos() {}

    /** Body of POST /api/forgot-password (step 1 of the page). */
    public record ForgotRequest(String email) {}

    /**
     * Answer of POST /api/forgot-password. It never says whether the e-mail has an account. requireMobile
     tells the page
     * whether step 2 must also ask for the registered mobile number (reset.require-mobile=true).
     */
    public record ForgotResponse(boolean requireMobile) {}

    /** Body of POST /api/reset-password (step 2). mobile is only needed when reset.require-mobile=true. */
    public record ResetRequest(String email, String newPassword, String mobile) {}

    /** Answer of POST /api/reset-password. */
    public record ResetResponse(String message) {}
}
```

springapp/src/main/java/com/examly/springapp/service/PasswordResetService.java

```
package com.examly.springapp.service;

import com.examly.springapp.config.RateLimiter;
import com.examly.springapp.dto.ResetDtos.ForgotResponse;
import com.examly.springapp.dto.ResetDtos.ResetResponse;
import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.exception.RateLimitExceededException;
import com.examly.springapp.exception.ResourceNotFoundException;
import com.examly.springapp.model.User;
import com.examly.springapp.repository.UserRepo;
import java.util.Locale;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

/**
 * "Forgot password" without e-mail: the person types the e-mail, then a new password, and the password is
 * changed at once.
 * <p>
 * <b>Security note.</b> Nothing proves that the person owns the e-mail, so anyone who knows an address can
 * change that
 * account's password. To limit the damage this service (1) never resets an ADMIN account, (2) limits
 * attempts per address and
 * per device, (3) revokes every refresh token after a reset, and (4) can ask for the account's registered
 * mobile number
 * as a second detail when {@code reset.require-mobile=true}. For a real product use the e-mailed link
 * instead.
 */
```

```

    */
@Service
public class PasswordResetService {
    private static final Logger log = LoggerFactory.getLogger(PasswordResetService.class);
    private static final long HOUR_MS = 3_600_000L;
    private static final String PASSWORD_POLICY = "^(?=.*[A-Za-z])(?=.*\\d){8,72}$";
    /** The mobile number stored for accounts created with Google sign-up; it is not a real secret, so it
cannot be used as proof. */
    static final String GOOGLE_PLACEHOLDER_MOBILE = "0000000000";

    private final UserRepo userRepo;
    private final PasswordEncoder encoder;
    private final RefreshTokenService refreshTokens;
    private final RateLimiter limiter;
    private final boolean requireMobile;

    @Autowired
    public PasswordResetService(UserRepo userRepo, PasswordEncoder encoder, RefreshTokenService
refreshTokens, RateLimiter limiter,
                               @Value("${reset.require-mobile:false}") boolean requireMobile) {
        this.userRepo = userRepo;
        this.encoder = encoder;
        this.refreshTokens = refreshTokens;
        this.limiter = limiter;
        this.requireMobile = requireMobile;
    }

    /** Step 1: checks the e-mail looks right and tells the page what step 2 needs. It does not reveal
whether an account exists. */
    public ForgotResponse start(String emailRaw, String clientIp) {
        guardRate("reset:start-ip:" + clientIp, 60, HOUR_MS, "Too many requests from this device. Try again
later.");
        cleanEmail(emailRaw);
        return new ForgotResponse(requireMobile);
    }

    /** Step 2: changes the password. */
    @Transactional
    public ResetResponse reset(String emailRaw, String newPassword, String mobileRaw, String clientIp) {
        String email = cleanEmail(emailRaw);
        guardRate("reset:ip:" + clientIp, 20, HOUR_MS, "Too many attempts from this device. Try again
later.");
        guardRate("reset:email:" + email, 10, HOUR_MS, "Too many attempts for this account. Try again in an
hour.");
        if (newPassword == null || !newPassword.matches(PASSWORD_POLICY)) {
            throw new InvalidRequestException("Password must be 8 to 72 characters with a letter and a
number");
        }
        User user = DatabaseOperationException.guard("loading a user", () ->
userRepo.findByEmail(email)).orElse(null);
        if (user == null) {
            throw new ResourceNotFoundException("No account found for this e-mail. Please register
first.");
        }
        if ("ADMIN".equalsIgnoreCase(user.getRole())) {
            throw new InvalidRequestException("Admin passwords cannot be reset here. Ask the platform
owner.");
        }
        if (requireMobile) {
            String stored = user.getMobileNumber() == null ? "" : user.getMobileNumber().trim();
            if (GOOGLE_PLACEHOLDER_MOBILE.equals(stored)) {
                throw new InvalidRequestException("This account was created with Google. Use Continue with
Google to log in.");
            }
            String given = mobileRaw == null ? "" : mobileRaw.replaceAll("\\D", "");
            if (!given.equals(stored)) {
                throw new InvalidRequestException("The mobile number does not match this account.");
            }
        }
        user.setPassword(encoder.encode(newPassword));
        DatabaseOperationException.guardVoid("saving the new password", () -> userRepo.save(user));
    }
}

```

```

        refreshToken.revokeAll(user.getUserId()); // every device must log in again with the new
password
        log.info("Password reset completed for user {}", user.getUserId());
        return new ResetResponse("Your password has been changed. You can log in now.");
    }

    private static String cleanEmail(String emailRaw) {
        String email = emailRaw == null ? "" : emailRaw.trim().toLowerCase(Locale.ROOT);
        if (email.isEmpty() || email.length() > 120 || !email.contains("@")) {
            throw new InvalidRequestException("Enter a valid e-mail address");
        }
        return email;
    }

    private void guardRate(String key, int limit, long windowMs, String message) {
        if (!limiter.tryAcquire(key, limit, windowMs)) {
            throw new RateLimitExceededException(message, limiter.retryAfterSeconds(key, windowMs));
        }
    }
}

```

springapp/src/main/java/com/examly/springapp/controller/PasswordResetController.java

```

package com.examly.springapp.controller;

import com.examly.springapp.dto.ResetDtos.*;
import com.examly.springapp.service.PasswordResetService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

/** Forgot password. Both endpoints are public (the person cannot log in); abuse is limited in the service
and the rate limit filter. */
@RestController
@RequestMapping("/api")
@Tag(name = "Forgot password", description = "Direct password reset: e-mail, then new password (no e-mail
is sent)")
public class PasswordResetController {
    private final PasswordResetService service;

    @Autowired
    public PasswordResetController(PasswordResetService service) {
        this.service = service;
    }

    @Operation(summary = "Step 1: check the e-mail", description = "Body: {email}. Answer: {requireMobile}.
Does not say whether the account exists.")
    @PostMapping("/forgot-password")
    public ResponseEntity<ForgotResponse> forgot(@RequestBody ForgotRequest body, HttpServletRequest
request) {
        return ResponseEntity.ok(service.start(body == null ? null : body.email(),
request.getRemoteAddr()));
    }

    @Operation(summary = "Step 2: set a new password", description = "Body: {email, newPassword, mobile?}.
mobile is needed only when reset.require-mobile=true.")
    @PostMapping("/reset-password")
    public ResponseEntity<ResetResponse> reset(@RequestBody ResetRequest body, HttpServletRequest request)
{
        return ResponseEntity.ok(service.reset(body == null ? null : body.email(), body == null ? null :
body.newPassword(), body == null ? null : body.mobile(), request.getRemoteAddr()));
    }
}

```